CY-5004 Secure Software Design

National University of Computer and Emerging Sciences, Islamabad

SecureWallet

Assignment 1: Mini Secure Fintech Wallet

Instructor	Muhammad Asim
Group members	Israr Ahmed (26I-7722)
	Burhan Ud Din (26I-7711)
Submission date	2 October 2026

Source code: `A1/` (Docker Compose project). Run instructions: `README.md`.

Contents

1 System overview **2**

2 Architecture **3**

3 Major assets **5**

4 Identified security weaknesses **5**

5 Possible impact of each weakness **7**

6 Proposed security controls **7**

7 Explanation of implemented controls **8**

8 Before-and-after security testing **13**

9 Security Design Table **17**

10 Security principles applied **19**

11 Individual student contributions **19**

12 Limitations and future improvements **20**

A Running the system **20**

B Security trace (presenter mode) **20**

1 System overview

SecureWallet is a small digital wallet. Registered customers hold a balance in Pakistani rupees and send it instantly to any other customer by username. It implements all six operations required by the brief (Table 1) and protects them with thirteen security controls. Each control answers one weakness identified in Section 4, has a before/after demonstration, and is covered by an automated test. All 39 tests pass.

Table 1: Required operations and how SecureWallet provides them.

Operation (brief §1)	Implementation
Create / use an account	Sign-up with full name, username and password. Each account gets exactly one wallet. Demo accounts are seeded: `ali`, `sara`, `bilal`, `hina` (customers) and `auditor`.
Log in	Username and password. The session ends after 5 minutes without activity. Transfers above a risk threshold also need a passkey (fingerprint, face or device PIN).
View balance	Balance card on the home screen, refreshed after every transfer.
Transfer funds	Send flow, laid out as a cheque: payee → amount (also written out in words) and memo → review, signed with the 6-digit transaction PIN → send. Sender is debited, receiver credited, one transaction recorded.
View transactions	Statement page: month sections with brought/carried-forward lines, sender, receiver, amount, date/time, status (completed or failed) and a running balance, with search and filters.
Log out	Sign out on this device, or on all devices from the Security page.

Technology. A React 18 single-page application is served by nginx. It talks to a FastAPI (Python 3.12) backend, which stores data in PostgreSQL 16. The three services run with Docker Compose. Money is stored as whole *paisa* (1 rupee = 100 paisa) in 64-bit integers, so no floating-point rounding can occur anywhere in a money path.

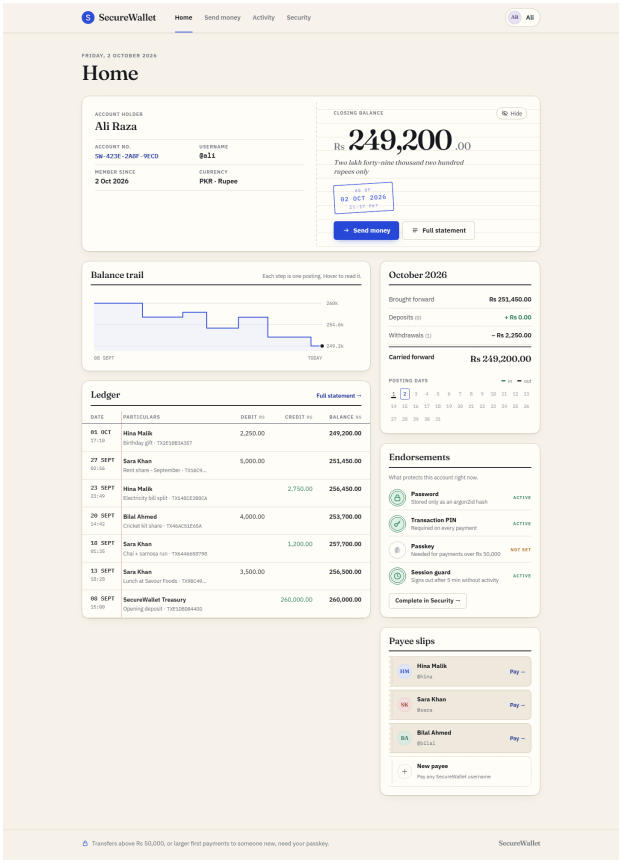
Interface. The interface is laid out as a bank passbook: the home page is the account page and ledger with a running balance on every line, sending money is writing a cheque signed with the transaction PIN, and the receipt is its counterfoil (Figure 1). Amounts are also written out in words in the Pakistani lakh/crore system, as on a real cheque, which helps a user notice a mistyped amount.

Roles. Customers hold and send money. A separate *auditor* role can read the audit log and run a ledger reconciliation but cannot move money.

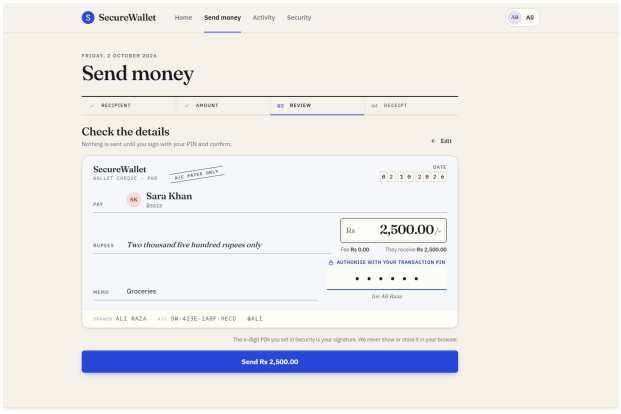
Presenter mode. For the live demonstration, a hidden presenter mode (Ctrl+Shift+D) adds two tools:

- a *security trace* panel showing every security check the server ran for each request, with real values;
- a *security lab* that switches each control off and on for the before/after tests in Section 8.

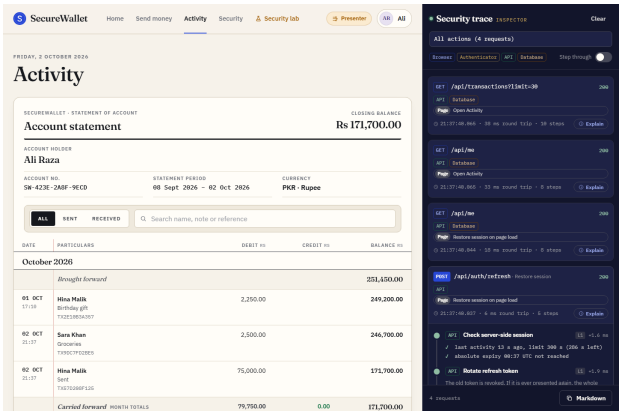
Both exist only when the server runs with `DEMO_MODE=true`. With the default `false`, the endpoints are not mounted at all.



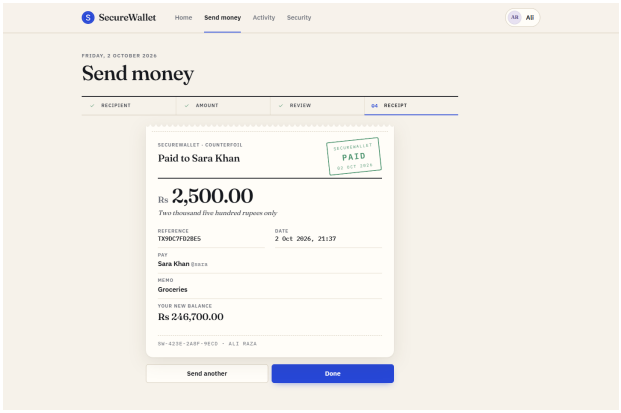
(a) Home: account page, balance trail, ledger with running balance, month reconciliation and security endorsements.



(b) Send money: the cheque, amount in words, signed with the transaction PIN.



(c) Statement with brought/carried-forward lines and running balance.



(d) Counterfoil after a completed transfer, stamped PAID.

Figure 1: The customer-facing application.

2 Architecture

Figure 2 shows the four components, the two Docker networks, and where each security control is enforced. The database has no published port and sits on an internal network with no route to the internet. Only the API can reach it. The API connects as a restricted database role, so the database itself enforces ownership and least privilege, even if the API code has a bug.

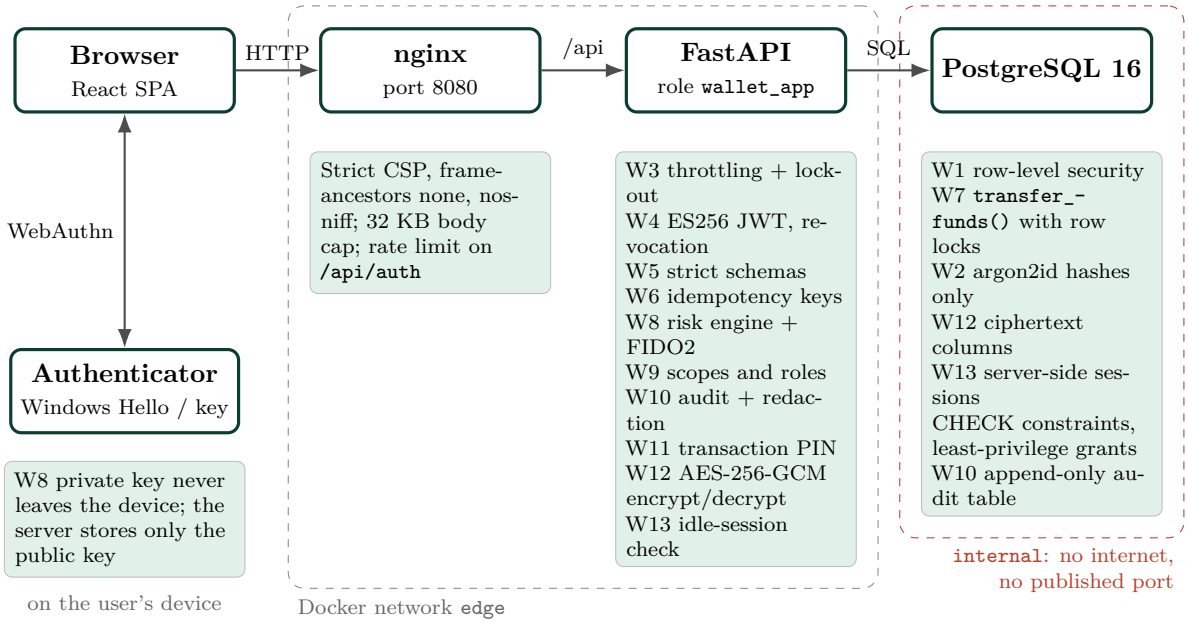


Figure 2: SecureWallet architecture and where each control (W1–W13) is enforced.

The controls follow the layered defence model from Lecture 2:

Table 2: Controls by defence-in-depth layer (Lecture 2).

Layer	Where	Controls
L1 Trust the actor	Auth API, authenticator	argon2id, throttling and lockout, ES256 tokens, revocation, refresh rotation, transaction PIN, idle timeout, passkey step-up
L2 Validate the action	API request pipeline	scopes and roles, ownership filter, strict schemas, idempotency, risk engine
L3 Protect the data	PostgreSQL	row-level security, least-privilege grants, atomic transfer function, CHECK constraints, AES-256-GCM encryption at rest
L4 Contain compromise	Docker, nginx	internal DB network, non-root read-only containers, dropped capabilities, CSP, secrets outside the repository
L5 Detect	Audit log	every outcome recorded, secrets redacted, append-only trigger
L6 Recover	Reconciliation	balances checked against the transaction history and the total supply

3 Major assets

Table 3: Assets that need protection.

ID	Asset	Why it matters
A1	User credentials (passwords, passkeys)	Whoever holds them controls the account and the money in it.
A2	Session tokens	A valid token lets the holder act as the user without the password.
A3	Wallet balances	The money itself. It must only change through a legitimate transfer.
A4	Transaction records	The legal and financial evidence of what happened; used for disputes and reconciliation.
A5	Customer identity data (name, username, history)	Personal data; exposing it breaches privacy and helps targeted fraud.
A6	Signing keys and secrets	The token signing key and database passwords; leaking them defeats authentication entirely.
A7	Audit log	The record that shows who did what; it must be complete and not editable.
A8	Service availability	Customers must be able to sign in and transfer when they need to.

4 Identified security weaknesses

We analysed the application with the questions in §4 of the brief and the threat landscape from Lecture 1 (BOLA, credential stuffing, transfer replay, insider misuse). Each question led to one concrete weakness in a naive implementation of the same features.

Table 4: Weaknesses, the question from the brief that exposed each, and the asset at risk.

ID	Question from the brief	Weakness	Asset
W1	Can one customer access another customer's information?	Broken object-level authorization (BOLA/IDOR): changing the wallet id in `/api/wallets/{id}` returns someone else's wallet.	A3, A4, A5
W2	Can credentials be exposed?	Passwords stored in a readable form.	A1
W3	Can unauthorized users perform sensitive operations?	Unlimited password guessing (brute force, credential stuffing).	A1
W4	Can unauthorized users perform sensitive operations?	A token whose payload is edited (e.g. `sub` changed) is accepted.	A2
W5	What happens if invalid input is submitted?	Negative, zero, fractional or oversized amounts, and extra fields such as `balance`, reach business logic.	A3
W6	Can transactions be manipulated?	A replayed or double-submitted transfer executes more than once.	A3, A4
W7	What happens if part of the system fails?	Parallel transfers read the same balance (race condition); a failure between debit and credit leaves partial state.	A3, A4
W8	Can unauthorized users perform sensitive operations?	A stolen password or session is enough to move a large amount.	A3
W9	Are privileged operations separated?	A customer can call auditor-only functions.	A7, A5
W10	Can important actions occur without being recorded?	Rejected actions are not recorded; logs may contain secrets.	A7, A1
W11	Can unauthorized users perform sensitive operations?	Anyone holding a live session (stolen token, or a screen left signed in) can pay money out; nothing checks it is still the owner.	A3
W12	Is sensitive information unnecessarily exposed?	Names, transfer notes and audit details sit in the database as plain text, readable from a dump, a backup or by a DBA.	A4, A5, A7
W13	Can unauthorized users perform sensitive operations?	A signed-in session never ends while the browser stays open, so an unattended screen stays usable.	A2, A3

5 Possible impact of each weakness

Table 5: Impact if each weakness were exploited, mapped to the CIA triad.

ID	Possible impact	CIA
W1	Any customer reads any other customer's balance and history: a privacy breach, and reconnaissance for targeted fraud.	C
W2	One database leak or stolen backup exposes every password. Password reuse spreads the damage to other services.	C
W3	Account takeover. Lecture 1 (slide 17) shows how 500,000 leaked credentials turned into thousands of drained accounts.	C, I
W4	An attacker impersonates any customer and moves their money.	I
W5	Sending −Rs 5,000 pulls money out of the recipient's wallet into the sender's.	I
W6	A double click or network retry charges the customer twice.	I
W7	Lost updates create or destroy money; the ledger no longer matches the balances.	I, A
W8	A phished password drains the whole wallet in one transfer.	I
W9	Customers read everyone's activity and could tamper with the evidence trail.	C, I
W10	Attacks go unnoticed, disputes cannot be resolved, and leaked logs reveal credentials.	I, C
W11	A hijacked or unattended session drains the wallet in many small transfers that stay under the passkey threshold.	I
W12	A stolen backup or database copy exposes customer names and what they paid for.	C
W13	Whoever walks up to an unlocked shared or office computer acts as the user.	C, I

6 Proposed security controls

The brief asks for a small number of well-justified controls. We chose one control per weakness, in each case the one that removes the weakness at its root rather than detecting it afterwards. Where a single point of failure would be dangerous (W1, W5, W9), we added a second, independent layer in the database. This extends existing logic rather than introducing a new technology.

Table 6: Proposed controls and why each was chosen.

ID	Control	Why this control
W1	PostgreSQL row-level security plus an ownership filter in the API	An ownership check in every API query is easy to forget on one new endpoint. RLS is enforced by the database on every query, so a single missing `WHERE` clause cannot leak data.
W2	argon2id password hashing	Salted and memory-hard: identical passwords give different hashes, and each guess costs the attacker 64 MiB of memory, so offline cracking of a leak is slow. Argon2id is the current OWASP recommendation.
W3	Per-IP rate limit plus per-account lockout with back-off	A per-IP limit alone is bypassed by botnets (Lecture 1, slide 17). A per-account lockout stops guessing against one account from any number of IPs.
W4	ES256-signed tokens with full claim validation and revocation	A signature makes any change to the token detectable. Pinning the algorithm blocks `alg:none` and key-confusion tricks. Revocation makes logout take effect immediately.
W5	Strict server-side validation, repeated in the database	The browser can be bypassed, so the server must validate. Repeating the check inside the transfer function and as CHECK constraints means a future API bug still cannot create money.
W6	Idempotency keys	The standard fix for retries (Lecture 2, slide 35): the second request with the same key returns the first result instead of executing again.
W7	One atomic transfer function with ordered row locks	Locks make parallel transfers queue; one database transaction makes debit and credit succeed or fail together. The API role has no direct UPDATE on balances, so there is only one way to move money.
W8	Risk-based FIDO2 step-up bound to the transfer	A passkey is phishing-resistant and cannot be stolen from the server (FIDO2 slides). Binding the challenge to the amount and payee means the signature approves only that transfer.
W9	Role-based scopes, checked in the API and in the database function	Least privilege and separation of duties: customers move money, auditors audit, and neither can do the other's job.
W10	Append-only audit log with redaction	Every outcome, including every rejection, is recorded with who, what and when, and nothing in the log can be edited or reveal a secret.
W11	6-digit transaction PIN on every transfer	A second factor the attacker does not get with the session. Stored only as an argon2id hash, three wrong entries lock it for 15 minutes, and weak PINs (123456, 111111) are refused.
W12	Application-level encryption at rest (AES-256-GCM)	The data is unreadable outside the application, and GCM's tag detects tampering. The key lives outside the database, so a database dump alone is useless.
W13	Server-enforced inactivity timeout	A browser-only timer can be bypassed, so the server records each session's last activity and refuses an idle session, including its refresh token.

7 Explanation of implemented controls

Each control is described with the file where it lives and what changed after it was added.

W1 Row-level security and ownership filter

The database has two roles, created in `db/init/01-roles.sh`. `wallet_owner` owns the tables. `wallet_app`, used by the API, is not the owner and has `NOBYPASSRLS`, so every policy applies to it. For

every request the API binds the verified user id to the database transaction (`backend/app/db.py`):

```
SELECT set_config('app.user_id', :sub, true);   -- true = SET LOCAL, cleared at COMMIT
```

The policies in `backend/app/sql/schema.sql` then filter every query:

```
ALTER TABLE wallets ENABLE ROW LEVEL SECURITY;
CREATE POLICY wallets_read ON wallets FOR SELECT
    USING (user_id = app_uid() OR app_role() = 'auditor');
CREATE POLICY transactions_read ON transactions FOR SELECT
    USING (sender_id = app_uid() OR receiver_id = app_uid() OR app_role() = 'auditor');
```

The API also filters by owner (`AND w.user_id = :caller` in `routers/wallet.py`). A foreign id returns `404`, the same response as a missing wallet, so ids cannot be enumerated. `SET LOCAL` matters because connections are pooled: the identity disappears at the end of the transaction and cannot leak into the next request.

W2 argon2id

`security/passwords.py` hashes with argon2id (`m=65536, t=3, p=2`, 16-byte random salt, 32-byte digest). When the username does not exist, the server still runs one verification against a dummy hash. This way response time does not reveal which usernames exist. A password policy rejects passwords under 10 characters, common passwords, and passwords containing the username.

W3 Throttling and lockout

`security/ratelimit.py` keeps a 60-second sliding window of 20 login attempts per IP. The `users` table counts failures: after 5, the account is locked for 60 seconds, doubling with each further failure up to 15 minutes. While locked, even the correct password is refused. Every failure returns the same message, "Incorrect username or password". nginx adds an outer limit on `/api/auth/`.

W4 Signed access tokens

`security/tokens.py` issues JWTs signed with ES256 (ECDSA on curve P-256). The key is generated on first start and kept in a Docker volume outside the image and the repository. Tokens last 15 minutes and carry `iss, sub, aud, iat, nbf, exp, jti`, role and scope. Verification:

- pins the algorithm to ES256 and checks the key id;
- validates every claim with a 30-second clock leeway;
- rejects any `jti` on the revocation list, which logout writes to.

The access token is kept only in browser memory, never in `localStorage`. Sessions are restored through a refresh token in an `HttpOnly; SameSite=Strict` cookie; only its SHA-256 hash is stored. Each use rotates it, and if a rotated token is ever presented again, the whole session family is revoked because it must have been copied.

W5 Server-side validation

`schemas.py` rejects unknown fields (`extra="forbid"`) and requires a strict integer amount, so `1.5`, `"100"` and an injected `balance` field are refused with `422`. `routers/transfers.py` checks that the amount is between Rs 0.01 and Rs 500,000, that the recipient is a customer, and that it is not a self-transfer. `transfer_funds()` repeats the amount check. Constraints `amount > 0`, `balance >= 0` and `sender <> receiver` are the last line of defence.

W6 Idempotency keys

Every transfer carries an `Idempotency-Key` header. The key, the SHA-256 of the request body, and the response are stored in `idempotency_keys` with primary key `(user_id, key)`. A concurrent duplicate waits on the unique index until the first request commits, then receives the stored response. The same key with a different body returns `409`. The front end generates one key per reviewed transfer, so a double click is safe.

W7 Atomic transfer function

All money moves through one PostgreSQL function, `transfer_funds()`, declared `SECURITY DEFINER`. It takes the sender from `app_uid()` rather than from a parameter. It then:

1. locks both wallets with `SELECT ... FOR UPDATE` in a fixed order (by user id), so opposite transfers cannot deadlock;
2. re-reads the balances under the lock;
3. debits, credits and inserts the transaction row inside one database transaction.

An insufficient balance is recorded as a *failed* transaction and nothing moves. The API role has no UPDATE right on `wallets` and no INSERT right on `transactions`, so this function is the only path that can change a balance.

W8 Risk engine and FIDO2 step-up

`security/risk.py` marks a transfer MEDIUM risk if it exceeds Rs 50,000, or if it is the first payment to a person and exceeds Rs 10,000. A MEDIUM transfer is not executed. It is stored as pending and the API returns `202` with WebAuthn options, whose challenge is bound to the transfer:

```
challenge = SHA-256( nonce(16 bytes) || canonical_json({pending_id, from, to, amount, note, exp})
    )
```

The browser asks the device's authenticator to sign it after local user verification. On `/confirm` the server:

1. recomputes the challenge from its own stored copy of the transfer;
2. checks `clientDataJSON` (type, challenge, origin);
3. checks `authenticatorData` (rpId hash, user-verified flag, signature counter);
4. verifies the ECDSA signature with the stored public key;
5. only then runs `transfer_funds()`.

A phishing site produces a different origin and rpId hash and fails. A replayed approval fails because the pending transfer is single-use. A customer without a passkey is asked to add one first. Figure 4 shows this verification in the security trace.

W9 Roles and scopes

Tokens carry a scope per role: `wallet:read transfer:write passkey:manage` for customers and `audit:read` for auditors (`deps.py`). Missing a scope gives `403`. The reconciliation function `reconcile_ledger()` checks the role again inside the database. So even with the API check switched off in the lab, a customer's reconciliation request is still refused.

W10 Audit trail

`audit.py` writes one row per outcome: success, failure, denial, lockout and lab switch. It writes on its own database connection, so the row survives a rollback of the failed request. Fields whose names match *password, token, secret, cookie, signature, credential* and similar are replaced by `[REDACTED]`. A trigger rejects every UPDATE and DELETE on `audit_log`. Error responses never include stack traces or SQL.

W11 Transaction PIN

Each customer sets a 6-digit PIN on the Security page, re-entering the account password to do so (`POST /api/tpin`, `routers/tpin.py`). The PIN is stored only as an argon2id hash in `users.tpin_hash`, never in the browser. `security/tpin.py` refuses repeated digits, ascending and descending runs, and common PINs.

`create_transfer` checks the PIN first, before idempotency, risk or any money logic:

- no PIN set: 403 `tpin_required`;
- wrong PIN: 403 `tpin_wrong` with the attempts left; the third wrong entry locks the PIN for 15 minutes (423 `tpin_locked`);
- correct PIN: the failure counter is reset.

The counter is committed before the error is returned, so a rollback cannot erase it. The PIN is excluded from the idempotency request hash and never written to the audit log. On the cheque, the PIN field is the signature line.

W12 Encryption at rest

`security/crypto.py` encrypts sensitive columns with AES-256-GCM and a fresh 96-bit nonce per value. The stored text is `enc:v1:<base64url(nonce || ciphertext || tag)>`, where `v1` is the key id, so keys can be rotated later. The 32-byte data key is generated on first start into the keys volume, outside the database and the image.

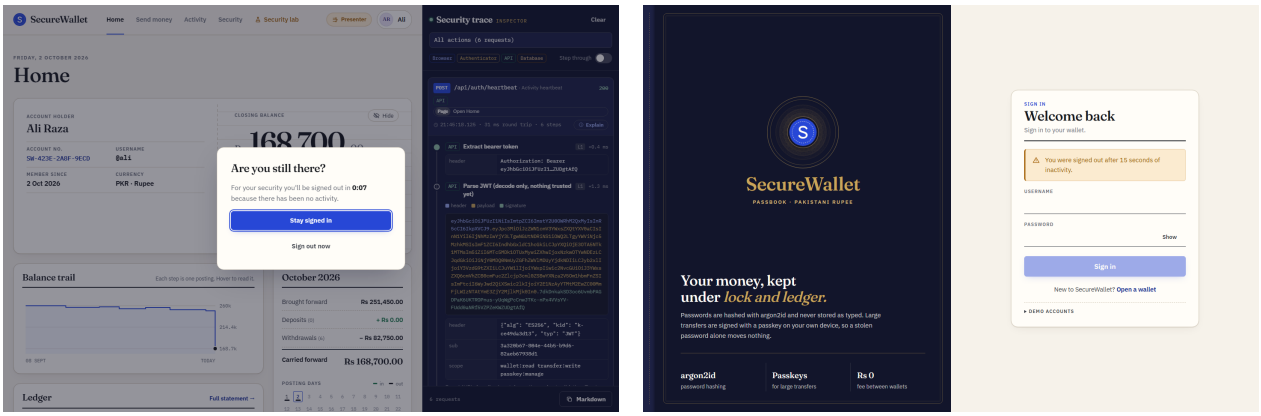
Each value is bound to its location through the GCM associated data, e.g. `users.full_name:<user id>`. A ciphertext copied to another row or column therefore fails to decrypt.

Encrypted columns: `users.full_name`, `transactions.note`, `pending_transfers.note`, `audit_log.detail`, and the stored idempotency responses. Balances and amounts stay as plain integers on purpose, because the database must add them up, lock them and check them inside `transfer_funds()` and `reconcile_ledger()`. Usernames stay plain because they are the sign-in lookup key.

W13 Inactivity timeout

Each sign-in creates a row in `sessions`; its id is the `sid` claim in every access token (`security/sessions.py`). On every request, `current_principal` loads the session and refuses it when it is revoked, idle for longer than the limit (5 minutes by default), or past its 8-hour absolute lifetime.

An expired session is marked revoked together with its refresh tokens, so refreshing cannot bring it back, and the response carries `code: session_idle`. Real user activity in the browser sends a throttled heartbeat. One minute before the limit the app asks “Are you still there?”, and at the limit it signs out and explains why on the sign-in page (Figure 3).



(a) Warning before the limit (demo limit of 15 seconds). **(b)** Signed out by the server, with the reason shown.

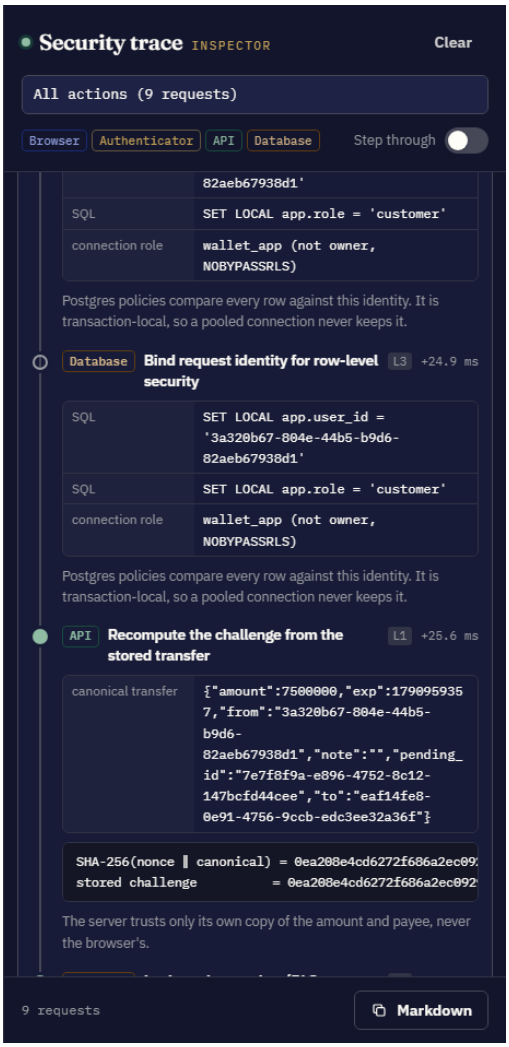
Figure 3: W13: the inactivity timeout.

Infrastructure

- **Containers:** they run as non-root with a read-only file system, all Linux capabilities dropped and `no-new-privileges`.
- **Secrets:** database passwords come from a git-ignored `.env`.
- **Browser policy:** nginx sends a strict Content-Security-Policy (`script-src 'self'`, no third-party origins, fonts self-hosted), `X-Frame-Options: DENY`, `nosniff` and `Referrer-Policy: no-referrer`.



(a) clientDataJSON checks and the authenticatorData byte map (rpIdHash, flags UP=1 UV=1, counter).



(b) The server recomputes the challenge from its own stored copy of the transfer (SHA-256 of nonce and canonical transfer) and binds the user id for row-level security.

Figure 4: Security trace of a Rs 75,000 transfer approved with a passkey. Values are from a real run.

8 Before-and-after security testing

Method. Following the brief's *Weakness → Demonstration → Control → Retest*, each control has a switch in the security lab (demo mode only). For each weakness:

1. We switch the control off and run the attack: *before*.
2. We switch it on and run the identical attack: *after*.

The switches for W1 and W5 change the real database: they enable or disable the row-level security policies, and drop or re-add the amount constraint. The same pairs are automated in `backend/tests/test_security.py`, which runs against the Docker stack. The final run passed **39 of 39 tests** in 5 minutes 27 seconds (19 core security tests, 7 transaction-PIN, 8 encryption-at-rest and 5 session tests).

Table 7: Before/after results (automated tests and live lab).

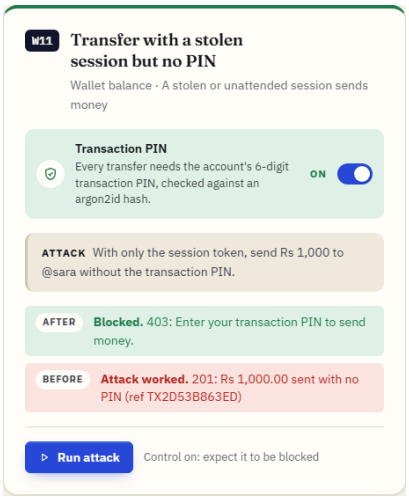
ID	Attack	Before (control off)	After (control on)
W1	Ali requests Sara's wallet by id	200: Sara's balance returned	404. With only the API filter off, still 404 because RLS blocks it
W2	Register, then dump the users table	`plain$open-sesame-2026`	argon2id hash (`$argon2id$v=19$m=65536...`)
W3	10 wrong passwords, then the right one	200: account taken over	429 from the 6th attempt; right password refused while locked
W4	Edit token `sub` to Sara, keep signature	200: signed in as Sara	401; `alg:none` also 401
W5	Send −Rs 5,000 to Sara	201: Ali's balance rises by Rs 5,000	422; also 1.5, `"100"`, 0, over-limit, extra `balance` field, self-transfer
W6	Same transfer 5 times at once, same key	charged more than once	charged once, one reference; changed body gives 409
W7	10 parallel Rs 1,000 transfers	balance falls by less than recorded; reconciliation fails	exact debit; reconciliation passes
W8	Rs 60,000 with only a session token	201: money moved	403 without passkey, 202 with one. Wrong origin 400, tampered challenge 400, replay 409
W9	Customer reads the audit log	200: log exposed	403; reconciliation 403 even with the API check off
W10	Failed login with a wrong password	Always on: the failure is audited with the username and never the password (verified by test).	
W11	Rs 1,000 transfer with only the session, no PIN	201: money sent	403 `tpin_required`; 3 wrong PINs give 423 lock
W12	Read a new user's row straight from the database	name stored as text: "Probe Person"	stored as `enc:v1:...` (AES-256-GCM); the app still shows the name
W13	Leave a session idle past the limit, then use its token	200: still usable	401 `session_idle`; refresh also refused

The functional tests in the same suite cover:

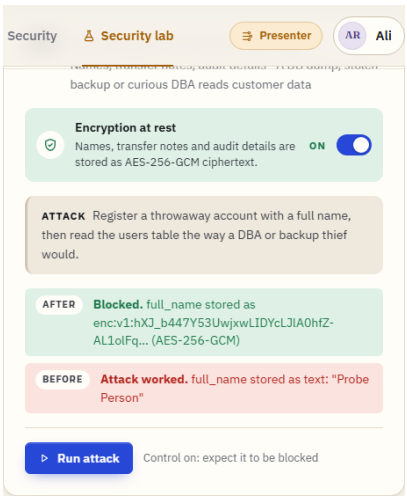
- the brief's example (Ali → Rs 2,000 → Sara): both balances change and both histories show the transaction;
- an insufficient-funds transfer recorded as *failed*;
- logout revoking the token;
- registration, including rejection of a duplicate username;
- refresh-token rotation with reuse detection.



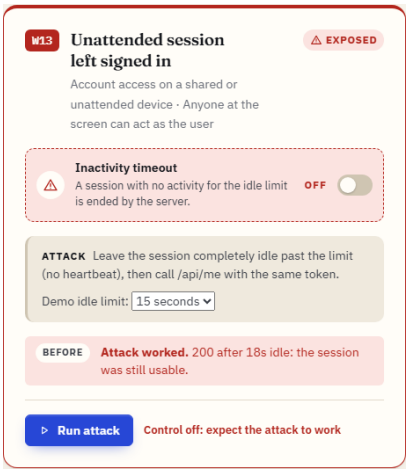
Figure 5: Security lab, W1: the *Before* run leaked Sara's balance (red); after both controls were switched back on, the same request was blocked with 404 (green). The trace panel on the right lists the requests, including the rejected `GET /api/wallets/...` with status 404.



(a) W11: without the PIN control a session alone sends money (201); with it, 403.



(b) W12: the stored name is plain text before, `enc:v1:...` ciphertext after.



(c) W13 with the control off: the idle session is still usable. With it on, see Figure 3.

Figure 6: Security lab, before/after runs for the controls added in this iteration.

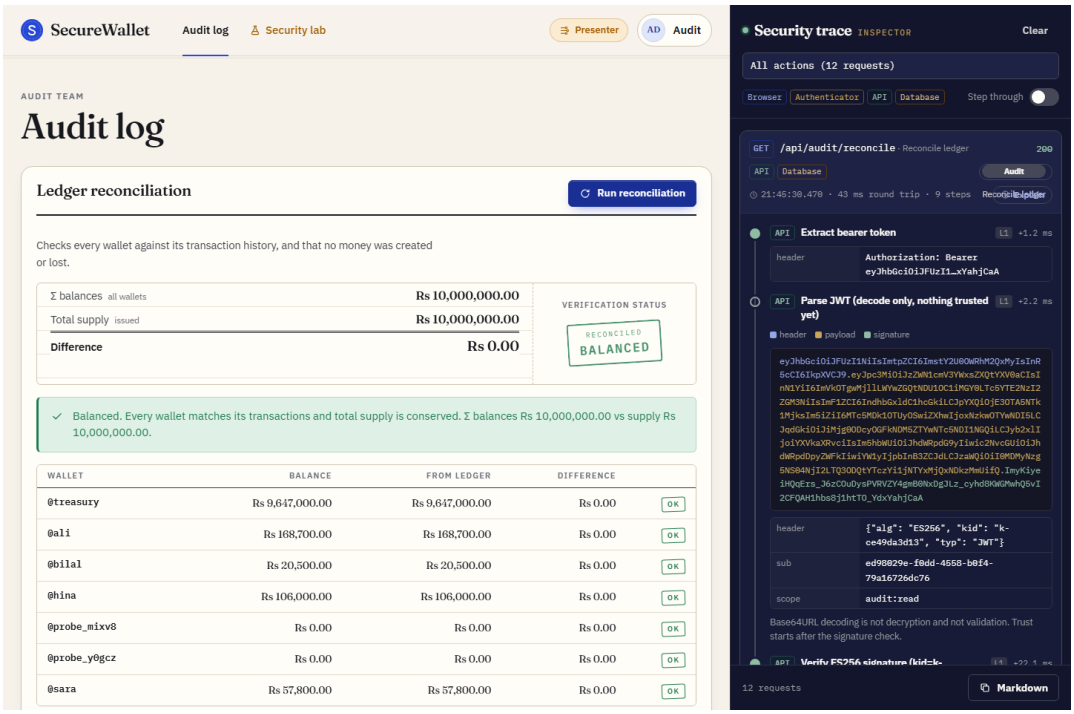


Figure 7: Auditor's register: reconciliation stamped BALANCED (every wallet equals its ledger and total supply is conserved), above the append-only audit trail.

9 Security Design Table

ID	Asset / Component	Identified weakness	Possible impact	Proposed control	How the control was implemented	Result
W1	Wallets, transactions / wallet API and database	BOLA: a changed id in the URL returns another customer's wallet	Privacy breach, fraud reconnaissance	Row-level security plus API ownership filter	RLS policies on `wallets` and `transactions`; API role `wallet_app` is not owner and has `NOBYPASSRLS`; `SET LOCAL app.user_id` per transaction; `AND user_id = :caller` in queries; 404 on mismatch	Before 200 with Sara's data; after 404; RLS alone still blocks
W2	Credentials / `users` table	Readable password storage	All passwords exposed by one leak	argon2id hashing	`passwords.py`: m=64 MiB, t=3, p=2, random 16-byte salt; dummy verify for unknown users; password policy	Stored value changes from plain text to an argon2id hash
W3	Login endpoint	Unlimited password guessing	Account takeover	IP rate limit and account lockout	`ratelimit.py`: 20 per minute per IP; lockout after 5 failures, 60 s doubling to 15 min; identical error message; nginx outer limit	Before: password found after 10 guesses; after: 429 from the 6th
W4	Access token / API	Edited token accepted	Impersonation of any customer	ES256 JWS, full claim validation, revocation	`tokens.py`, `deps.py`: algorithm pinned, key id, iss/aud/exp/nbf/iat/jti required; revocation list on logout; rotating refresh cookie with reuse detection	Forged token: 200 before, 401 after
W5	Balance / transfer API	Invalid amounts and extra fields reach business logic	Money pulled from another wallet	Strict validation in API and database	Pydantic `extra="forbid"`, strict integer; Rs 0.01–500,000; re-check in `transfer_funds()`; CHECK constraints	−Rs 5,000: 201 before, 422 after
W6	Transfer API	Replay and double submit	Customer charged twice	Idempotency keys	`Idempotency-Key` header; `(user_id, key)` primary key; stored response replayed; 409 if body differs	5 identical requests: charged once
W7	Balances and ledger / database	Race condition, partial failure	Lost updates, money created or destroyed	Single atomic transfer function with row locks	`transfer_funds()` SECURITY DEFINER; `FOR UPDATE` in user-id order; one DB transaction; API role cannot UPDATE balances	10 parallel: lost updates before; exact and reconciled after

ID	Asset / Component	Identified weakness	Possible impact	Proposed control	How the control was implemented	Result
W8	High-value transfers	Stolen password or session enough to move large sums	Wallet drained	Risk engine plus FIDO2 step-up bound to the transfer	`risk.py`: over Rs 50,000, or new payee over Rs 10,000; 202 pending; challenge = SHA-256(nonce, transfer); origin, rpId, UV, counter and ECDSA signature verified with py_webauthn	201 before; 202 then passkey after; phishing origin and replay rejected
W9	Audit functions	Customer reaches privileged operations	Exposure and tampering of evidence	Role scopes checked in API and database	Scopes per role in the token; `require_role`; role check inside `reconcile_ledger()`	Customer: 200 before, 403 after; DB still refuses
W10	Audit log	Unrecorded rejections, secrets in logs	No accountability, credential leak	Append-only audit log with redaction	`audit.py` writes every outcome on its own connection; key-based redaction; UPDATE/DELETE trigger; no stack traces returned	Failed login audited without the password
W11	Transfers / transfer API	A live session alone can pay	Hijacked or unattended session drains the wallet	6-digit transaction PIN on every transfer	`tpin.py`, `routers/tpin.py`: argon2id hash, weak-PIN list, 3 tries then 15-minute lock, checked first in `create_transfer`	No PIN: 201 before, 403 after
W12	Names, notes, audit details / database	Stored as readable text	Dump or backup exposes customer data	AES-256-GCM encryption at rest	`crypto.py`: `enc:v1` format, per-row AAD, key in the keys volume; encrypt on write, decrypt on read	DB shows ciphertext; app shows plaintext
W13	Sessions / API	Session never ends while the tab is open	Unattended screen used by someone else	Server-enforced idle timeout	`sessions.py`: `sessions` table, `sid` claim, idle and absolute limits checked on every request and on refresh; heartbeat and warning in the browser	Idle token: 200 before, 401 after

10 Security principles applied

Table 9: How the design relates to the Unit 1 principles (brief §8).

Principle	Where it is applied
Confidentiality	RLS and ownership checks (W1); argon2id (W2); encryption at rest (W12); lookups return only name and username, never ids or balances.
Integrity	Signed tokens (W4); validation in three places (W5); idempotency (W6); atomic transfers with locks (W7); transfer-bound passkey approval (W8).
Availability	Rate limits protect the login and the API; the database refuses to start a transfer it cannot finish; the stack restarts automatically.
Authentication	argon2id passwords, lockout, ES256 tokens, the transaction PIN on every transfer (W11), the inactivity timeout (W13), passkeys for high-risk actions.
Authorization	Scopes per role, object ownership in the API and the database.
Least privilege	The API's database role cannot update balances or insert transactions directly; auditors cannot move money; customers cannot audit.
Secure defaults	Demo mode is off by default, so the lab and trace do not exist; every lab switch starts in the secure position.
Separation of responsibilities	Customer and auditor roles; the database owner role is used only for migrations.
Defence in depth	payments need a session, then the PIN (W11), then a passkey above the limit (W8); W1 (API filter and RLS), W5 (schema, API rules, DB function, constraints), W9 (API and database role checks).
Reduction of unnecessary exposure	The database has no published port; responses carry minimal fields; errors carry no internal details.
Logging and accountability	Append-only audit log of every outcome, with secrets redacted (W10).
Resilience	Atomic transactions leave no partial state; idempotent retries are safe; reconciliation detects inconsistencies.

11 Individual student contributions

Table 10: Contributions of each group member.

Student	Major contribution
Israr Ahmed (26I-7722)	Architecture and Docker setup; database schema, row-level security and the atomic transfer function (W1, W7); authentication: password hashing, throttling, signed tokens and refresh sessions (W2–W4); encryption at rest (W12); audit log (W10); security lab and the automated before/after tests.
Burhan Ud Din (26I-7711)	Transfers, server-side validation and idempotency (W5, W6); passkeys and the risk engine (W8); roles and the auditor desk (W9); transaction PIN (W11); inactivity timeout (W13); the passbook front end and security trace; security analysis and this report.

Every member can explain the whole application, as the brief requires.

12 Limitations and future improvements

- **No TLS in the local stack.** Traffic between the browser and nginx is plain HTTP, which WebAuthn allows only on `localhost`. A deployment would terminate TLS 1.3 at nginx.
- **Keys in a Docker volume.** The token signing key sits in a Docker volume. Production would keep it in a key-management service or HSM and rotate it using the existing key id.
- **In-memory rate limiter.** The per-IP limiter lives in the API process. Several API instances would need a shared store such as Redis. The per-account lockout is already in the database.
- **No real funding source.** Money enters only through seeded treasury deposits. A top-up from a bank would need a signed, replay-protected payment callback (Lecture 1, slide 19).
- **Simple risk engine.** It uses amount and payee history only. Velocity limits, device recognition and new-login alerts are natural next steps.
- **Stolen session cookie: analysed, not demonstrated.** The refresh token sits in an `HttpOnly; SameSite=Strict` cookie, so page scripts cannot read it and other sites cannot send it; it is rotated on every use, with reuse detection that revokes the whole session; and the idle timeout (W13) limits how long a copy stays useful. A copied cookie replayed directly, however, would still be accepted until one of those triggers. The planned fix, binding the session to a non-extractable key held by the browser (DPoP-style device binding), and a lab demonstration of the replay were not built in this iteration.
- **Encryption key and balances.** The data key lives in the Docker keys volume; production would use a KMS or HSM with envelope encryption. Balances and usernames are not encrypted, by design (Section 7, W12).
- **PIN space.** A 6-digit PIN has one million values. Online guessing is stopped by the 3-try lock, but if the database and the hashes were stolen, argon2id only slows an offline search; it cannot make one million guesses infeasible.
- **Lab reset clears the audit log.** This is an intentional demo-only owner action. It does not exist when demo mode is off.

A Running the system

```
cp .env.example .env          # set the three database passwords
docker compose up --build     # open http://localhost:8080
# demo accounts: ali, sara, bilal, hina, auditor  (password Wallet@2026!)

# tests (API exposed on 127.0.0.1:8000)
docker compose -f docker-compose.yml -f docker-compose.dev.yml up -d --build
pytest backend/tests -v
```

B Security trace (presenter mode)

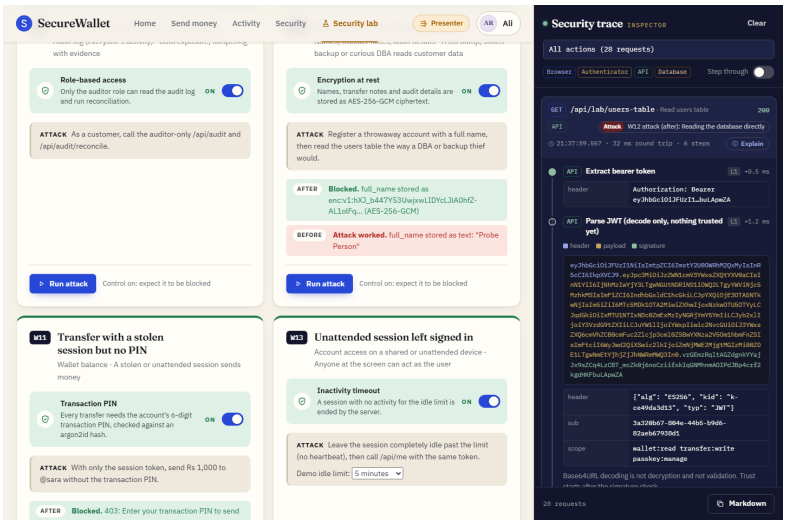
When presenter mode is on, the front end sends the header `X-Security-Trace: 1`. The backend records each security decision it makes for that request and returns the steps with the response. Examples: the token signature check with its ECDSA arithmetic, the claims, the RLS identity binding, validation rules, the risk decision, the SQL of the transfer, and the audit record.

The panel adds several aids for the demonstration:

- It groups requests by the user action that caused them, and a filter shows a single action (Figure 8).

- Each request is tagged with the components it touched and the time it was sent.
- An *Explain* button describes each request in plain language.

Without demo mode the header is ignored and nothing is collected.



(a) Security lab with the trace panel: each request is tagged with the action that caused it (here the W12 attack), the time it was sent and the parts of the system it touched.

Figure 8: The security trace used during the live demonstration.